Informasjon om eksamenen

Emnekode: DAT120
Emnenavn: Grunnleggende Programmering
År og semester: 2023 Høst
Eksamensdato: 13. desember 2023
Klokkeslett: 09.00 - 13.00

Tillatte hjelpemiddel:

- Godkjent kalkulator
- to A4-ark med egne notater på eksamen. Disse notatene kan være enten håndskrevne eller maskinskrevne. Du kan skrive på begge sidene av A4-arkene.

Faglig ansvarlig: Erlend Tøssebro
Telefonnummer: +47 48107119

Om eksamenen:
Du bør lese instruksjonene før du starter med å løse eksamensoppgavene!

For oppgaver som krever programmering: Det viktigste er at du viser hvordan en løsning kan programmeres i Python. Enkle skrivefeil i navn på funksjoner vil ikke trekke ned. Å bruke et annet navn på en funksjon vil heller ikke trekke ned så lenge det går klart fram hva du prøver å gjøre og hvilken funksjon du prøver å kalle.

Det er lov å skrive hjelpemetoder og hjelpeklasser utover det oppgaven ber om hvis du mener det er den beste måten å løse oppgaven på.

Evaluering:
Merk at all kode blir evaluert ikke bare på om den virker men også på hvor ryddig og lesbar den er. Du vil bli trukket for uleselig kode selv om den gjør alt oppgaven ber om.

Oversikt over Python kommandoer og nøkkelord

Nøkkelord

Nøkkelord (engelsk Keyword) er reserverte ord med fast betydning i Python

Definisjoner

def <navn på funksjon eller metode>(<Liste med parametere>):

> Definerer en funksjon. Hvis denne står inne i blokka til en klasse så er det en metode og første parameter skal være self.

class <navn på klassen>:

> Definerer en klasse

Import av kode

from <navn på fil eller pakke> **import** <navn på funksjon eller klasse>

> Importerer navngitte funksjoner eller klasser fra en pakke eller fil. Du kan oppgi flere med komma mellom. Du kan skrive * for å importere alle.

import <navn på fil eller pakke> **as** <alias>

> Importerer en pakke eller ei Python fil. Den importerer Python filer gjennom å kjøre dem. Den siste delen «as <alias>» er valgfri, og brukes for å gi pakken eller fila et kortere navn enn den ellers ville ha hatt.

Kontrollstrukturer

break

> Avbryter løkka som den står i. Den må stå inne i en for-løkke eller ei while-løkke

continue

> Hopper tilbake til starten av løkka den står i og tar neste verdi for en for-løkke.

for <variabel> **in** <liste av verdier>:

> <blokk, som utføres en gang for hver verdi i lista, med variabel satt lik verdien den utføres for nå>

if <betingelse>:

> <blokk, som utføres hvis betingelsen er sann (True)>

elif <betingelse>:

> <blokk, som utføres hvis alle foregående betingelser i denne if-setningen er usanne (False) men denne betingelsen er True>

else:

<blokk, som utføres hvis alle betingelsene i denne if-setningen er usanne>

while <betingelse>:

<blokk, som utføres gang etter gang helt til betingelsen er usann>

with <ressurs> **as** <variabelnavn>:

<blokk som bruker ressursen. Ressursen lukkes alltid når blokken fullfører, uansett hvordan den fullfører. I DAT120 har det bare vært filer som har vært brukt som slike ressurser>

Boolske operatorer

Disse opererer på de boolske verdiene False og True, og kan brukes for å lage mer kompliserte betingelser for if- og while-setninger.

a **and** b: Er sann bare hvis både a og b er sanne.

a **or** b: Er sann hvis minst en av a og b er sanne. Er usann bare hvis både a og b er usanne.

not a: Er sann hvis a er usann og usann hvis a er sann.

Matematiske operatorer

a + b: Fungerer som vanlig matematisk + for tall. Slår sammen to strenger eller lister

a – b: Matematisk subtraksjon

a * b: For tall gjør denne multiplikasjon. For strenger og lister lager den en ny streng eller liste med b antall duplikater av a.

a ** b: Opphøying, a^b.

a / b: Flyttallsdivisjon. Returnerer alltid et flyttall.

a // b: Heltallsdivisjon. Kutter desimalen hvis resultatet hadde vært et flyttall. 11 // 3 gir resultatet 3

a % b: Modulo. Returnerer resten etter en heltallsdivisjon. 11 % 3 gir resultatet 2 siden det høyeste tallet under 11 som går opp i 3 er 9, og 11 er to høyere enn 9. 9 % 3 gir resultatet 0.

Sammenlikningsoperatorer

a == b	Er a lik b?
a != b	Er a ulik b?
a < b	Er a mindre enn b?
a > b	Er a større enn b
a <= b	Er a mindre enn eller lik b?
a >= b	Er a større enn eller lik b?

Innebygde kommandoer

Dette er kommandoer (funksjoner) som er bygget inn i Python språket.

chr(<tall>)

Returnerer tegnet som er representert med dette tallet

float(<verdi>)

Prøver å konvertere verdien til et flyttall. Kan konvertere både heltall og strenger. Strengen må inneholde bare et flyttall, ellers risikerer du en ValueError.

input(<spørsmål>)

Skriver ut spørsmålet og ber deretter brukeren om å skrive inn en verdi. Returnerer verdien brukeren skreiv inn som en streng.

int(<verdi>)

Prøver å konvertere verdien til et heltall. Kan konvertere både flyttall og strenger. Kutter desimalen til et flyttall. En streng må inneholde bare et heltall, ellers risikerer du en ValueError.

len(<samling>)

Returnerer antall elementer i samlingen som et heltall. For et dictionary er det antall nøkler som er lagt inn.

max(<liste med sammenliknbare verdier>)

Returnerer det høyeste elementet i lista

min(<liste med sammenliknbare verdier>)

Returnerer det laveste elementet i lista

open(<filnavn>, <modus>, encoding=<navn på kodetabell>)

Åpner fila med oppgitt navn og returnerer en referanse til denne fila. Modus har default «r» for lesing, men kan også være «w» for skriving. Man kan bruke den navngitte parameteren encoding for å si hvilken kodetabell tekst er kodet på. Filreferansen som returneres kan man bruke til å lese og skrive til fila. Skriving skjer med write() metoden. Lesing skjer med read() eller readline() metodene, eller at du bruker filreferansen som lista med verdier i en for-løkke.

ord(<streng med ett tegn>)

Returnerer tallkoden til dette tegnet

print(<verdi>)

Skriver ut en streng. Konverterer andre verdier til strenger på samme vis som **str**

range(start, slutt, steglengde)

Lager en slags liste av verdier. Denne kan brukes i for-løkker men er ikke ei Python liste. Lista starter med verdien «start», slutter før verdien «slutt» og går «steglengde» langt mellom hver verdi. Range-funksjonen tar bare heltall. Ønsker du å bruke flyttall for for eksempel steglengde, må du bruke numpy.arange eller numpy.linspace i stedet. Range med en parameter oppgir bare slutt, med start 0. Range med to parametere oppgir start og slutt, med stengelengde 1.

round(<flyttall>, <valgfri: antall desimaler>)

Runder av et flyttall. Uten den andre parameteren returnerer den et heltall. Ellers runder den av til det oppgitte antallet desimaler og returnerer et flyttall.

str(<verdi>)

Konverterer verdien til en streng. Er verdien et objekt gjør den dette ved å kalle __str__ metoden til objektet.

sum(<liste med tall>)

Regner ut summen av alle elementene i lista.

Strengmetoder

Disse metodene kan man kalle på et str objekt:

find(<delstreng>, <valgfri: start>, <valgfri: slutt>)

Leiter etter en delstreng av strengen. Returnerer indeksen i strengen til første tegn i delstrengen hvis delstrengen fins i strengen. Ellers returnerer den -1. Parameterne start og slutt er indeksene hvor den skal starte og slutte med å leite.

lower()

Returnerer strengen med alle bokstavene gjort om til små bokstaver.

replace(<delstreng den skal se etter, a>, <delstreng den skal erstatte med, b>)

Returnerer en ny streng der alle forekomster av delstrengen a er erstattet av delstrengen b.

split(<streng, valgfri>)

Splitter strengen basert på den oppgitte delstrengen, eller på whitespace (åpenrom, tabulatorer og linjeskift) hvis denne parameteren ikke er oppgitt. Returnerer ei liste med strenger.

upper():

Returnerer strengen med alle bokstavene gjort om til store bokstaver.

Formatering av strenger

Formateringskoder

Disse formateringskodene kan brukes i f-strenger for å formatere dem

Kode	Betydning	Eksempel
d	Heltall i titallssystemet (det normale tallsystemet). Standard for heltall. Antall siffer den skal sette av plass til oppgis rett før.	Tall = 7 Print(f«tallet er {tall:d}») Print(f«tallet er {tall:10d}»)
c	Konverterer heltallet ti et tegn med den nåværende kodetabellen	
x	Heltall i det heksadesimale tallsystemet	
b	Heltall i det binære tallsystemet	
f	Flyttall med komma-notasjon. Tar to parametere. I eksemplet skal den sette av 5 siffer hvorav 2 bak komma. Merk at komma-tegnet regnes som ett av sifrene!	Flyttall = 7.3 Print(f«tallet er {flyttall:5.2f}»)
e	Flyttall på eksponentialform (e-notasjon)	
%	Flyttall som prosent. Ganger tallet med 100 og viser det med et prosent-tegn etter. Parametere som for «f»	

Spesialtegn

Man kan bruke disse kodene i en streng for å lage spesialtegn

Kode	Betydning
\n	Linjeskift
\t	Tabulator
\	Backslash
\"	Anførselstegn (slik inkluderer du anførselstegn i strenger)

Metoder for lister

Man kan bruke disse metodene på lister

append(<verdi>)

Legger verdien til på slutten av lista

index(<verdi>)

Leiter etter verdien og returnerer første indeks hvor den fins. Lager en AttributeError hvis verdien ikke fins i lista.

insert(<indeks>, <verdi>)

Setter inn verdien på oppgitt indeks. Alle elementer som lå på eller etter denne indeksen blir forskjøvet ett hakk bakover i lista.

remove(<verdi>)

Fjerner første forekomst av verdien i lista, alle elementer bak blir flyttet ett hakk fram. Lager en ValueError hvis verdien ikke fins i lista.

sort()

Sorterer lista. Alle elementene i lista må være sammenliknbare med hverandre, ellers får du en TypeError. Returnerer ingen ting. Modifiserer lista.

Spesialmetoder brukt i temaet klasser og objekter

Disse metodene brukes av selve Python språket for å håndtere objekter av klassen.

__init__(self, <parameterliste>)

Konstruktøren for klassen. Denne kalles når du lager nye objekter av klassen. Den tar de parameterne som du definerer. Når man lager objektene så må man da oppgi disse parameterne.

__repr__(self)

Lager og returnerer en strengrepresentasjon mer ment til internt bruk

__str__(self)

Lager og returnerer en strengrepresentasjon av objektet, ment for brukere å lese.

Matplotlib

Her er en oversikt over funksjoner i Matplotlib som er brukt i dette faget.

import matplotlib.pyplot as plt

Standard setning for å importere matplotlib

plt.subplot(<Antall rader>, <Antall kolonner>, <Hvilken av sub-figurene du skal starte på nå>)

Brukes for å tegne opp flere sub-plott i ett vindu. Antall rader og antall kolonner bør være like for alle kallene til plt.subplot, mens den siste parameteren bør være ulik. Den siste parameteren starter å telle på 1, og går først horisontalt, deretter vertikalt.

plt.plot(<liste av x-koordinater>, <liste av y-koordinater>, < tegnestil>, label=<hvilken kurve er dette?>)

Tegner et standard linje-plott. Listene bør være like lange. Den starter i punktet (x[0], y[0]), tegner ei linje til (x[1], y[1]), deretter til (x[2], y[2]) og så videre til den når slutten av listene. Tegnestil og label er valgfrie parametere. Tegnestil er en streng som inneholder ett eller flere av disse tegnene: «o» for å tegne sirkler for hver verdi, «-» for å tegne linje mellom verdiene, «*» for å tegne stjerner for hver verdi, og flere.

Flere plot-kommandoer etter hverandre kan brukes for å tegne flere linje-plott i samme figur.

plt.bar(<liste av verdier langs x-aksen>, <liste av høydene for hver stolpe>, color=<liste av farger for stolpene>)

Tegner et stolpediagram. Parameteren «color» er valgfri. De to listene må ha indekser som korresponderer med hverandre. Så høyden til stolpen til den første verdien langs x-aksen må ligge i første element i lista av høyder.

plt.pie(<Liste av verdier>, labels=<Liste av etiketter, en for hver verdi>)

Tegner et kakediagram. Den regner ut summen av verdiene og regner ut vinklene slik at vinkelen til hvert kakestykke korresponderer med hvor mange prosent av summen hver verdi representerer.

plt.hist(<liste av verdier>, <Antall stolper>)

Tegner et histogram av verdier.

plt.title(<Tittel>)

Gir plottet en tittel, som tegnes øverst, sentrert.

plt.xlabel(<beskrivelse>)

Gir en beskrivelse av hva x-koordinatene er evt. enhet langs x-aksen. Tegnes under x-aksen.

plt.ylabel(<beskrivelse>)

Gir en beskrivelse av hva y-koordinatene er evt. enhet langs y-aksen. Tegnes vertikalt til venstre for y-aksen.

plt.legend()

Ber matplotlib lage en boks som inneholder label for alle plottene i figuren

plt.grid(True)

Viser et rutenett

plt.savefig(<Filnavn>)

Lagrer figuren. Filtypen avgjøres av etternavnet til filnavnet du oppgir, for eksempel .pdf.

plt.show()

Viser figuren

Numpy

Her er en oversikt over funksjoner i numpy som er brukt i dette faget

import numpy as np

Importerer numpy og gir den dens valnlige alias

np.arange(<start>, <slutt>, <steg>)

Lager og returnerer en numpy array som starter med tallet «start», slutter rett før den når «slutt» og med «steg» mellom tallene. Arrayen blir så lang som den trenger å være. I motsetning til den innebygde range-funksjonen så aksepterer np.arange flyttall som parametere.

np.array(<liste>)

Lager og returnerer en numpy array som inneholder de samme elementene som den oppgitte lista.

np.linspace(<start>, <slutt>, <antall>)

Lager og returnerer en numpy array som starter med tallet «start», slutter med tallet «slutt» og med «antall» elementer. Elementene har lik avstand mellom hverandre. Merk at denne tar med verdien «slutt», i motsetning til numpy.arange og den innebygde range funskjonen.

np.ones(<form>)

Lager og returnerer en numpy array med oppgitt form og hvor alle elementene er 1. Hvis form er et heltall lager den en endimensjonal array med den lengden. For eksempel med form=7 så lager den en endimensjonal array som er 7 lang. Hvis form er et tuppel lager den en flerdimensjonal array (matrise) med antall dimensjoner lik antall elementer i tuppelet og antall elementer i hver retning lik tallene i tuppelet. For eksempel med form=(3, 4) lager den en todimensjonal matrise som er 3 ganger 4 stor.

np.zeros(<form>)

Lager og returnerer en numpy array med oppgitt form og hvor alle elementene er 0. Form er som for np.ones.

Turtle Graphics

Her er en oversikt over funksjoner i Turtle Graphics som er brukt i dette faget.

import turtle

Importerer Turtle Graphics

turtle.forward(<lengde>)

Går forover. Tegner hvis penna er nede. Penna starter nede

turtle.backward(<lengde>)

Går bakover. Tegner hvis penna er nede. Penna starter nede

turtle.right(<vinkel i grader>)

Snur seg den oppgitte vinkelen til høyre

turtle.left(<vinkel i grader>)

Snur seg den oppgitte vinkelen til venstre

turtle.penup()

Tar opp penna

turtle pendown()

Tar ned penna

turtle.circle(<radius>)

Tegner en sirkel med oppgitt radius. Posisjonen til skilpadda nå er der den starter å tegne, og vil derfor være langs sirkelen og ikke i midten.

turtle.speed(<fart>)

Setter farta til skilpadda. Speed 0 er så raskt som mulig. Speed 1 er så tregt som mulig. Så øker hastigheten opp til speed 10. Deretter er den fast.

turtle.fillcolor(<farge>)

Setter fyllfargen. Fargen er en streng med et fargenavn på engelsk.

turtle.pencolor(<farge>)

Setter farge på penna

turtle.pensize(<tjukkelse>)

Setter tjukkelse på penna

turtle.begin_fill()

Starter å tegne mønsteret som skal fylles

turtle.end_fill()

Fyller mønsteret som er tegnet med fyllfargen

turtle.done()

Sier fra at den er ferdig med å tegne. Brukes i script for å sørge for at turtle-vinduet forblir oppe selv etter at scriptet er ferdig.

Git

Basis håndtering av repo-et

Kommando	**Bruk**
clone <lenke til	Lager et lokale repo og en lokal filstruktur som en kopi av et eksternt

eksternt repo>	repo.
branch <navn>	Lager en ny branch i ditt lokale repo
checkout <navn på branch>	Bytter til angitt branch. Oppdaterer filstrukturen din med innholdet av denne branchen
push	Sender endringer (gjort med en commit) til det eksterne report som denne er klonet fra. Brukes til å oppdatere Github med endringer. For å oppdatere Github med en ny branch, bruk push --set-upstream origin <navn på branch>
Pull	Henter endringer gjort på tjeneren ned til lokalt repo. Må muligens bruke git pull origin main for å hente ned ny main-branch etter en merge konflikt.

Basis håndtering av filer

Kommando	**Bruk**
add <mappenavn>	Registrerer endringer og nye filer før en commit. Kan brukes for å legge til nye filer og å si fra om oppdateringer du har gjort på eksisterende filer. Du kjører typisk git add på hele mapper, men den kan også kjøres på enkeltfiler. Kjør alltid git add før en git commit. Du kan fint kjøre flere add før en enkelt commit, også på de samme filene.
status	Sier fra hvilke filer som er registrert for å committe, det vil si hvilke filer du har lagt til med git add siden sist commit.
diff	Kan brukes på flere måter, den enkleste, uten ekstra parametere, sjekker filene dine for endringer og sier fra hva du burde legge til med add før en commit
Commit -m <melding>	Oppdaterer ditt lokale repository med endringene du har lagt til med add, og legger til meldingen slik at andre som bruker repositoryet kan se hva endringen er
restore <filnavn>	Setter den oppgitte fila tilbake slik den var ved siste commit

ℹ Kode-eksempler

Se vedlagte .pdf fil.

1 Tallsystemer

Hva blir det binære tallet 11101 i det vanlige titallssystemet? _____ . 4%

Hva blir tallet 98 som binærtall? _____ . 4%

Maks poeng: 8

2 Variabler og datatyper

I øving 1 oppgave f skulle du lage et program som svarte på følgende oppgave:

"Man oppgir vanligvis størrelsen til en TV eller dataskjerm som antall tommer langs diagonalen. For å finne ut om det er plass til TV-en der du tenker å ha den trenger du ofte å vite bredden og høyden til TV-en. Skriv et Python-script som regner ut bredde og høyde til en skjerm basert på lengden til diagonalen. Brukeren skal oppgi denne diagonale lengden. Anta at skjermen er på 16:9 format (forholdet mellom bredde og høyde er 16:9, en skjerm som er 16 brei er 9 høy). Scriptet skal skrive ut lengden og bredden til skjermen. Du kan skrive ut lengde og bredde i samme enhet som brukeren oppgir diagonalen i (typisk tommer)."

Under svaralternativene og i den vedlagte .pdf fila er det seks forslag til svar på denne oppgaven, som alle er varianter av løsningsforslaget. Du skal krysse av for alle forslagene som er riktige svar på oppgaven.

Poeng: 3 poeng for hvert riktige kryss, -3 poeng for hvert feil kryss. Maksimalt 9 poeng, minimum 0 poeng.

Velg ett eller flere alternativer

- ☐ Forslag 1
- ☐ Forslag 2
- ☐ Forslag 3
- ☐ Forslag 4
- ☐ Forslag 5
- ☐ Forslag 6

```
# Forslag 1
skjermstorrelse_str = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse = float(skjermstorrelse_str)
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse/nevner
bredde = 16*faktor
hoyde = 9*faktor
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")
```

```
# Forslag 2
skjermstorrelse_str = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse = float(skjermstorrelse_str)
bredde = 16*faktor
hoyde = 9*faktor
```

```
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse/nevner
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")

# Forslag 3
skjermstorrelse_str = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse = float(skjermstorrelse_str)
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse/nevner
hoyde = 9*faktor
bredde = 16*faktor
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")

# Forslag 4
skjermstorrelse = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse_float = float(skjermstorrelse)
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse_float/nevner
hoyde = 9*faktor
bredde = 16*faktor
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")

# Forslag 5
skjermstorrelse = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse_float = float(skjermstorrelse)
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse_float/nevner
hoyde = 9*faktor
bredde = 16*faktor

# Forslag 6
skjermstorrelse = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse_float = float(skjermstorrelse)
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse/nevner
hoyde = 9*faktor
bredde = 16*faktor
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")
```

Maks poeng: 9

3 Basis programmering

Skriv et Python-script som regner ut volumet til ei kjegle. Brukeren skal oppgi radius til bunnen av kjeglen (r i formelen) og høyden til kjeglen (h i formelen). Etter å ha regnet ut volumet skal scriptet skrive dette ut. 5%

Formelen til volumet for ei kjegle: $volum = \frac{1}{3}\pi r^2 h$

Skriv ditt svar her

```
1
```

Maks poeng: 5

4 Løkker og lister

Del 1: Gitt følgende Python-script:

```
liste = list()
for i in range(5):
    liste.append(2*i)
print(liste)
```

Hva vil lista "liste" inneholde? Skriv inn hvert element i riktig rekkefølge i tekstfeltene her. Dersom du mener det er for mange tallfelt, skriv "-" i de tallfeltene du mener er overflødige.

[ ] , [ ] , [ ] , [ ] , [ ] , [ ]

Del 2: Gitt følgende Python-script

```
liste = list()
liste.append(1)
liste.append(1)
for i in range(2, 5):
    liste.append(liste[i-1] + liste[i-2])
print(liste)
```

Hva vil lista "liste" inneholde? Skriv inn hvert element i riktig rekkefølge i tekstfeltene her. Dersom du mener det er for mange tallfelt, skriv "-" i de tallfeltene du mener er overflødige.

[ ] , [ ] , [ ] , [ ] , [ ] , [ ] , [ ]

Poeng: 1% per korrekt tall.

Maks poeng: 13

5 Flervalgsoppgave

Deloppgave 1:
Gitt følgende oppgavetekst for en funksjon:

Lag en funksjon som lar brukeren skrive inn et heltall som er større enn eller lik 0. Hvis brukeren ikke skriver inn et lovlig heltall skal brukeren få beskjed om det og få prøve på nytt. Du trenger bare å teste for om tallet er under 0, du trenger ikke å teste for om det brukeren skriver inn ikke er et tall.

Her er det oppgitt fire alternative svar på denne oppgaven. Tre av dem er korrekte, det siste er feil. Vis hvilket svar som er feil i valgboksen under.

```
# Alternativ 1
def les_positivt_heltall1(beskjed):
    tall = int(input(beskjed))
    while tall < 0:
        print("Du må skrive inn et ikke-negativt heltall!")
        tall = int(input(beskjed))
    return tall

# Alternativ 2
def les_positivt_heltall1(beskjed):
    tall = int(input(beskjed))
    while tall < 0:
        print("Du må skrive inn et ikke-negativt heltall!")
    tall = int(input(beskjed))
    return tall

# Alternativ 3
def les_positivt_heltall1(beskjed):
    streng = input(beskjed)
    tall = int(streng)
    while tall < 0:
        print("Du må skrive inn et ikke-negativt heltall!")
        streng = input(beskjed)
        tall = int(streng)
    return tall

# Alternativ 4
def les_positivt_heltall1(beskjed):
    fortsetter = True
    while fortsetter:
        tall = int(input(beskjed))
        if tall < 0:
            print("Du må skrive inn et ikke-negativt heltall!")
        else:
            fortsetter = False
    return tall
```

Velg hvilken av løsningene som ikke er korrekt:

- Alternativ 3
- Alternativ 4
- Alternativ 1
- Alternativ 2

Deloppgave 2:

Gitt samme oppgave som i del 1 bortsett fra at den også må håndtere at brukeren skriver inn noe som ikke er et tall. Her er fire alternative løsninger. Tre av dem er feil, et av dem er korrekt. Velg hvilket av alternativene som er korrekt i valgboksen under.

```
# Alternativ 1
def les_positivt_heltall_med_eh(beskjed):
    fortsetter = True
    while fortsetter:
        try:
            tall = int(input(beskjed))
            fortsetter = False
        except ValueError:
            print("Du må skrive inn et heltall!")
            continue
        if tall < 0:
            print("Du må skrive inn et ikke-negativt heltall!")
    return tall

# Alternativ 2
def les_positivt_heltall_med_eh(beskjed):
    fortsetter = True
    while fortsetter:
        try:
            tall = int(input(beskjed))
        except ValueError:
            print("Du må skrive inn et heltall!")
            continue
    if tall < 0:
        print("Du må skrive inn et ikke-negativt heltall!")
    else:
        fortsetter = False
    return tall

# Alternativ 3
```

```
def les_positivt_heltall_med_eh(beskjed):
    fortsetter = True
    while fortsetter:
        try:
            tall = int(input(beskjed))
        except ValueError:
            print("Du må skrive inn et heltall!")
            continue
        if tall < 0:
            print("Du må skrive inn et ikke-negativt heltall!")
            fortsetter = False
    return tall

# Alternativ 4
def les_positivt_heltall_med_eh(beskjed):
    fortsetter = True
    while fortsetter:
        try:
            tall = int(input(beskjed))
        except ValueError:
            print("Du må skrive inn et heltall!")
            continue
        if tall < 0:
            print("Du må skrive inn et ikke-negativt heltall!")
        else:
            fortsetter = False
    return tall
```

Velg hvilken av de fire løsningene som er korrekt

- Alternativ 3
- Alternativ 2
- Alternativ 1
- Alternativ 4

Poeng: 5% per korrekt svar, 0% for feil eller manglende svar.

Maks poeng: 10

6 Vurdering av kode

Denne oppgaven inneholder en serie med deloppgaver. Hver deloppgave inneholder en oppgavetekst og et forslag til kode som kanskje løser oppgaven slik den er beskrevet. Din oppgave er for hver av disse deloppgavene:

1: Gjør denne koden det oppgaveteksten i deloppgaven ber om? Hvis ja, skriv "ja" i tekstfeltet under den deloppgaven.
2: Hvis koden ikke gjør det oppgaveteksten ber om, skriv hva som er feilen i tekstfeltet under deloppgaven og beskriv hvordan å lage en korrekt løsning.

Score: Opp til 5% for hver deloppgave.

Deloppgave 1
Oppgavetekst: Du ønsker å finne ut hvor lang tid det tar å kjøre en gitt vegstrekning. Lag et Python-script som leser inn avstanden i meter og fartsgrensa i kilometer pr. time. Scriptet skal regne ut hvor lang tid i sekunder det tar å kjøre denne strekningen med antakelse om at man kjører med en hastighet lik fartsgrensa, og skrive ut resultatet.

Forslag:

```
tid_i_sekunder = 0

def tid_aa_kjore(avstand_i_meter, fart_i_kilometer_per_time):
    tid_i_sekunder = avstand_i_meter / (fart_i_kilometer_per_time * 1000 / 3600)

avstand_i_meter = float(input("Avstand i meter: "))
fartsgrense = float(input("Fartsgrense i km/t: "))
tid_aa_kjore(avstand_i_meter, fartsgrense)
print(tid_i_sekunder)
```

Skriv ditt svar her

Deloppgave 2
Oppgavetekst: Skriv en funksjon som tar inn to lister. Funksjonen skal lage ei ny liste som bare inneholder de elementene i liste 1 som også finnes i liste 2.

Forslag:

```
def med_i_begge(liste1, liste2):
    resultat = list()
    for element in liste1:
        if element in liste2:
            resultat.append(element)
    return resultat
```

Skriv ditt svar her

Deloppgave 3

Oppgave: Skriv en funksjon som tar inn to lister med tall. Funksjonen skal lage ei ny liste hvor elementet på posisjon i i resultatet skal være summen av elementene ved posisjon i i de to listene som kommer inn. Hvis listene som kommer inn er av ulik lengde skal resultatet ha lengde som den korteste av dem.

Forslag:

```
def sum_liste(liste1, liste2):
    resultat = list()
    for i in range(len(liste1)):
        resultat.append(liste1[i] + liste2[i])
    return resultat
```

Skriv ditt svar her

Deloppgave 4

Oppgave: Gitt klassene Punkt og Linje. Klassen Linje representerer ei linje som består av ei liste med punkter hvor det er rette linjer mellom nabopunkter i lista. Lag en funksjon som leser ei linje tekst i Well-Known Text format (forkortet WKT), oppgitt under, og lager et Linje objekt fra dette formatet. WKT-formatet er et format for å lagre geometriske objekter slik at de kan leses av både mennesker og maskiner. Linjer i WKT-format er representert som strenger i Python.

Eksempel på WKT-format for ei linje:

LINESTRING(5 10,10 10,10 15,5 20)

Dette representerer ei linje som starter i punktet med x-kooridinat 5 og y-koordinat 10, heretter representert som (5, 10), går gjennom punktene (10, 10), (10, 15) og slutter i punktet (5, 20).

Generell beskrivelse av formatet:

LINESTRING(<punkt>, <punkt>, ..., <punkt>)

Ei linje må inneholde minst to punkter, men kan inneholde så mange som trengs. Hvert punkt er oppgitt med sine x- og y-koordinater med åpenrom mellom seg. Merk at x- og y- koordinatene er bare skilt med dette åpenrommet.

```
# Gitt, du kan anta at denne er korrekt
class Punkt:
    def __init__(self, x_koordinat, y_koordinat):
        self.x_koordinat = x_koordinat
        self.y_koordinat = y_koordinat
```

```
# Gitt, du kan anta at denne er korrekt
class Linje:
    def __init__(self, startpunkt, sluttpunkt):
        self.punktliste = list()
        self.punktliste.append(startpunkt)
        self.punktliste.append(sluttpunkt)

    def legg_til_punkt(self, punkt):
        self.punktliste.append(punkt)


# Forslag til løsning
def les_linje_fra_WKT(wkt_streng):
    if wkt_streng.find('LINESTRING') == -1:
        return None
    punktliste = list()
    punktliste_streng = wkt_streng[11:-1].split(',')
    for punkt in punktliste_streng:
        koordinater = punkt.split(' ')
        x_koordinat = float(koordinater[0])
        y_koordinat = float(koordinater[1])
        punktliste.append(Punkt(x_koordinat, y_koordinat))
    return Linje(punktliste[0], punktliste[1])
```

Skriv ditt svar her

Maks poeng: 20

7 Pascal's trekant

Pascal's trekant er en trekant av tall på følgende format:

1
1, 1
1, 2, 1
1, 3, 3, 1
1, 4, 6, 4, 1
osv.

Hver linje starter og slutter på 1. Tallene som er i mellom er summen av tallet på samme posisjon i linja over og tallet i posisjonen rett til venstre i linja over. Så for eksempel i linje 5 så er første tall 1. Andre tall i linje 5 er summen av første og andre tall i linje 4, som er 1 + 3 = 4. Tredje tall i linje 5 er summen av andre og tredje tall i linje 4, som er 3 + 3 = 6.

Skriv et Python script som lar brukeren skrive inn antall linjer av Pascal's trekant den skal skrive ut og så skriver ut det antall linjer.

Poeng: 10%

Skriv ditt svar her

```
1
```

Maks poeng: 10

8 Filer og exceptions

Du har to filer med elementer som er sammenliknbare. Den første fila heter "grupper.txt" og inneholder elementer på følgende format:

4, 7, 2, 3
12, 17, 45
3, 11
15, 16, 23

Hver linje representerer ei gruppe med elementer. I dette tilfellet er hvert element et heltall.

Den andre fila heter "elementer.txt" og inneholder ett element på hver linje:

3
4
17
34

Skriv et Python-script som finner hvilke av linjene i den første fila hvor minst ett av elementene i gruppa fins i den andre fila. Python-scriptet skal skrive ut minimum linjenummeret til hver linje i den første fila hvor den finner minst ett av elementene i den andre fila.

I eksempel-filene som er oppgitt vil linje 1, 2 og 3 være riktig svar. I den første linja er tallene 3 og 4 med i den andre fila. I den andre linja er tallet 17 med i den andre fila. I den tredje linja er tallet 3 med i den andre fila. Ingen av tallene i den fjerde linja er med i den andre fila.

Merk at ditt Python-script skal fungere også for andre filer enn eksempel-fila oppgitt her så lenge filene er i samme format.

Poeng:

- Korrekt lesing av filene 6%
- Korrekt håndtering av feil, inkludert tomme linjer. Tomme linjer skal den hoppe over. Du trenger bare å håndtere feil i lesing av en av de to filene. 4%
- Finner korrekt hver linjei den første fila som har minst ett element i den andre fila og skriver dette ut 6%

Skriv ditt svar her

```
1
```

Maks poeng: 16

9 Dictionaries

Gitt ei liste av strenger på følgende format:

[«1984: 3», «1987: 6», «1979: 2», «1984: 2», …]

Hvor det før kolonet er et årstall.

Deloppgave 1: Skriv en funksjon som tar inn ei slik liste og lager og returnerer et dictionary med en nøkkel for hvert år og med summen av tallene bak kolonet for det året som verdi. Merk at det vil være flere innslag for mange av årene i lista. Det ikke er innslag i lista for hvert år og innslagene ikke nødvendigvis er sortert etter år. (6%)

Deloppgave 2: Skriv en funksjon eller et script som skriver ut dictionariet som blir returnert i forrige deloppgave sortert etter år. Hint: Du kan hente ut et liste med nøkler fra et dictionary som ligger i variabelen "dictionary" med linja oppgitt under: (3%)

nokkel_liste = list(dictionary.keys())

Skriv ditt svar her

```
1
```

Maks poeng: 9

Document 3
Attached



Kode-eksempler til eksamen i DAT120

Kontrollstrukturer

Er dette tallet for en måned gyldig?

```
tall_streng = input("Skriv inn et tall for en måned: ")
maaned = int(tall_streng)
if maaned < 1 or maaned > 12:
    print("ugyldig måned")
else:
    print("Gyldig måned")
```

Løsninger av en andregradslikning

```
import math

a = int(input("Skriv inn tallet a: "))
b = int(input("Skriv inn tallet b: "))
c = int(input("Skriv inn tallet c: "))

verditest = b**2 - 4*a*c

if verditest > 0:
    losning1 = (-b + math.sqrt(verditest))/(2*a)
    losning2 = (-b - math.sqrt(verditest))/(2*a)
    print(f"Likningen har to løsninger: {losning1} og {losning2}")
elif verditest == 0:
    losning = (-b)/(2*a)
    print(f"Likningen har en løsning: {losning}")
else:
    print("Likningen har ingen løsninger")
```

Lar brukeren skrive inn flere linjer tekst og avslutter når brukeren skriver inn ei tom linje

```
teksten = ""
tekstlinje = input("Skriv inn første linje: ")
while tekstlinje != "":
    teksten += tekstlinje + "\n"
    tekstlinje = input("Skriv inn neste linje: ")
print("Den endelige teksten ble:")
```

```
print(teksten)
```

Regner ut bokstavkarakter fra prosentscore, avslutter hvis brukeren skriver inn en negativ score

```
fortsetter = True

while fortsetter:
    prosentscore = int(input("Skriv inn en prosentscore: "))
    if prosentscore < 0:
        fortsetter = False
        break
    if prosentscore >= 90:
        print("A")
    elif prosentscore >= 80:
        print("B")
    elif prosentscore >= 60:
        print("C")
    elif prosentscore >= 50:
        print("D")
    elif prosentscore >= 40:
        print("E")
    else:
        print("F")
```

Regner ut fakultet

```
tall = int(input("Skriv inn tallet du ønsker fakultet av: "))
while tall < 0:
    print("Fakultet for negative tall finnes ikke!")
    tall = int(input("Skriv inn tallet du ønsker fakultet av: "))

resultat = 1
for i in range(1, tall+1):
    print(i)
    resultat *= i
print("Etter for-løkka er ferdig")
print(resultat)
```

For-løkke som teller nedover

```
for i in range(10, 0, -1):
    print(i)
print("Etter for-løkka er ferdig")
```

Nøstet for-løkke som skriver ut en firkant av ASCII stjerner

```
hoyde = int(input("Høyde: "))
bredde = int(input("Bredde: "))

for j in range(hoyde):
    for i in range(bredde):
        print("*", end="")
    print()
```

Funksjoner

Areal og omkrets av en sirkel

```
import math

def areal_sirkel(radius):
    areal = math.pi*radius*radius
    return areal


def omkrets_sirkel(radius):
    return 2.0*math.pi*radius


radius_streng = input("Skriv inn radius til sirkelen: ")
radius_bruker = float(radius_streng)
areal_global = areal_sirkel(radius_bruker)
print(f"Arealet ble: {areal_global:8.2f}")
omkrets = omkrets_sirkel(radius_bruker)
print(f"Omkretsen ble: {omkrets:8.2f}")
```

Funksjon med flere parametere: Skriv ut en ASCII firkant av tegn

```
def skriv_firkant(hoyde=5, bredde=5, tegn="*"):
    for j in range(hoyde):
        for i in range(bredde):
```

```
            print(tegn, end="")
        print()

hoyde = int(input("Høyde: "))
bredde = int(input("Bredde: "))

skriv_firkant(2, 4)
print("\n ny firkant \n")
skriv_firkant(hoyde, bredde, "#")
print("Ferdig")
```

Filer og exceptions

Les inn ei tekstfil og skriv ut innholdet

```
filnavn = input("Hvilken fil skal leses? ")
fila = open(filnavn, "r", encoding="UTF8")
for linje in fila:
    print(linje, end="")
fila.close()
```

Lar brukeren skrive inn flere linjer tekst og lagrer dette til fil. Avslutter når brukeren skriver inn ei tom linje

```
tekstlinje = input("Skriv inn første linje: ")
fila = open("tekstfil.txt", "w", encoding="UTF8")
while tekstlinje != "":
    fila.write(tekstlinje + "\n")
    tekstlinje = input("Skriv inn neste linje: ")
fila.close()
```

Eksempel på håndtering av feil brukerinput

```
fortsetter = True
while fortsetter:
    try:
        tall = int(input("Skriv inn tallet du ønsker fakultet av:
"))
        if tall < 0:
            print("Fakultet for negative tall finnes ikke!")
        else:
```

```
            fortsetter = False
    except ValueError:
        print("Du må skrive inn et tall!")

resultat = 1
for i in range(1, tall+1):
    print(i)
    resultat *= i
print("Etter for-løkka er ferdig")
print(resultat)
```

Eksempel på with-setningen og håndtering av feil med filer

```
filnavn = input("Hvilken fil skal leses? ")
try:
    with open(filnavn, "r", encoding="UTF8") as fila:
        summen = 0
        for linje in fila:
            summen += int(linje)
        print(f"Summen ble: {summen}")
except FileNotFoundError:
    print("Finner ikke denne fila!")
except ValueError:
    print("Fila har feil format")
except:
    print("En annen feil har oppstått")
```

Samlingsobjekter

Lister

```
# Lager ei tom liste
liste = list()

# Lager ei liste med oppgitte elementer, her tallene 1, 2, 3, 4 og 5
liste2 = [1, 2, 3, 4, 5]

# Skriver ut lengden til lista
print(len(liste2))
```

```
# Legger til elementet 4 i lista "liste"
liste.append(4)

# Legger til elementet 7 i lista "liste2"
liste2.append(7)

# Henter ut elementet med indeks 3 og skriver det ut. Merk
# at Python teller fra 0, første elementet i lista har indeks 0!
print(liste2[3])

# Elementene i ei liste kan være hva som helst
liste.append("En streng")

# Inkludert andre lister
liste.append([9, 8, 7])

# Kan overskrive verdien på en bestemt indeks med tilordning
liste2[2] = 9

# Negative indekser teller fra slutten
print(liste2[-1])

# Setter inn elementet 10 på indeks 2. Dette forskyver
# alle elementer som er etterpå en plass lengre ut i lista
liste2.insert(2, 10)

# For lister inni lister, bruk flere indekser etter hverandre.
# For eksempel denne, som kan leses
# Hent ut elementet med indeks 2 i liste. Hent ut elementet
# med indeks 1 i lista som ligger i indeks 2 i liste.
print(liste[2][1])

# Lager ei liste hvor innholdet er lik indeksene
liste_indekser = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,
15]

# Lager ei ny liste so består av elementene fra og med
# indeks 3, til men ikke med indeks 8, i liste_indekser
```

```
test = liste_indekser[3:8]
print(test)

# Starter fra starten av lista og går til men ikke med indeks 8
print(liste_indekser[:8])

# Starter på indeks 8 og går til slutten av lista
print(liste_indekser[8:])

# Starter på indeks 1 og tar med bare annethvert element.
# Stopper rett før den når indeks 10
print(liste_indekser[1:10:2])

# Finner det minste elementet
print(min(liste_indekser))

# Finner det største elementet
print(max(liste_indekser))

liste3 = [1, 2, 3, 4, 5]
liste4 = [9, 10, 11, 12]

# Kan bruke + operatoren til å slå sammen lister
liste5 = liste3 + liste4

# Kan bruke * operatoren til å repetere ei liste. Denne lager
# ei liste med elementene 0,1 repetert 5 ganger
liste6 = [0, 1]*5

# Skriver ut indeks til første element som inneholder verdien 10
print(liste4.index(10))

# Fjerner første forekomst av elementet 9 fra lista
liste5.remove(9)

# Fjerner elementet som ligger på indeks 5 fra lista
del liste5[5]
```

```
# Sorterer lista. Denne modifiserer lista slik at den er sortert
liste5.sort()

# Reverserer lista slik at den får motsatt rekkefølge
liste2.reverse()
```

Bruk av lister i ei for-løkke

```
liste = [1, 3, 5, 7, 9]

for element in liste:
    element = element*2
    print(element)

print(liste)

element = input("Skriv inn et tall for å sjekke om det er med i
lista: ")
element = int(element)
if element in liste:
    print(f"{element} er med i lista")
else:
    print(f"{element} er ikke med i lista")

for indeks, element in enumerate(liste):
    print(f"Element {element} ligger på indeks {indeks}")
```

List comprehensions

```
# Lager en liste med kvadratene av tallene fra 1 til 10
liste = list()
for verdi in range(1, 11):
    liste.append(verdi**2)
print(liste)

# Lager samme liste med en list comprehension
liste2 = [verdi**2 for verdi in range(1, 11)]
print(liste2)

# Lager en liste med kvadratene av oddetallene fra 1 til 20
```

```python
liste3 = [verdi**2 for verdi in range(1, 20) if verdi%2 == 1]
print(liste3)

# Lager samme liste uten list comprehension
liste4 = list()
for verdi in range(1, 20):
    if verdi%2 == 1:
        liste4.append(verdi**2)
print(liste4)
```

Numpy arrayer

```python
import numpy as np

# Lager en numpy array med 5 0-ere
nullere = np.zeros(5)

# En for-loop med steglengde 0.1
for i in np.arange(1, 5, 0.1):
    print(f"{i:5.2f}")

# Lager en numpy array med 4 elementer. Det første er 1,
# det siste er 10, og de to mellom er jevnt spredd mellom
# 1 og 10
test = np.linspace(1, 10, 4)
for element in test:
    print(test)

# Lager en numpy array fra ei liste
oddetall = np.array([1, 3, 5, 7, 9])

# Lager ei liste fra en numpy array
liste = list(test)

# Kan lage matrise med å oppgi et tuppel til np.zeros. Ekseplet
# lager en 4x3 matrise
matrise = np.zeros((4, 3))
```

```
# Lan lage en matrise fra ei liste av lister så lenge
# de indre listene alle er like lange
matrise2 = np.array([[1, 2, 3], [2, 2, 2], [3, 2, 1]])
```

Mengder

```
mengde1 = set()
mengde1.add("Henrik")
mengde1.add("Nils")
mengde1.add("Olav")
mengde1.add("Anders")
mengde1.add("Henrik")

mengde1.add("Åsmund")

for element in mengde1:
    print(element)
print("\n Mengde 2: ")
mengde2 = set()
mengde2.add("Nils")
mengde2.add("Andreas")
mengde2.add("Tomas")
mengde2.add("Olav")
mengde2.add("Mads")
for element in mengde2:
    print(element)


# Mengden av alle som er med i minst en av de to
m3 = mengde1.union(mengde2)
print("\n Union: ")
for element in m3:
    print(element)

# Mengden av alle som er med i begge
m3 = mengde1.intersection(mengde2)
print("\n Snitt: ")
for element in m3:
    print(element)
```

```
# Mengden av alle som er med i mengde1 men ikke i mengde2
m3 = mengde1.difference(mengde2)
print("\n m1 minus m2: ")
for element in m3:
    print(element)
```

Dictionaries

```
# Ordteller for tekstfil:
#   Les inn ei tekstfil
#   Finn alle ordene
#   Tell antall forekomster av hvert ord

ordliste = dict()
with open("oving_1_rein_tekst.txt", "r", encoding="UTF8") as fila:
    for linje in fila:
        ordene = linje.split()
        for ordet in ordene:
            ordet = ordet.lower()
            if ordet in ordliste:
                teller = ordliste[ordet]
                teller += 1
                ordliste[ordet] = teller
            else:
                ordliste[ordet] = 1
    for ordet in ordliste:
        print(f"Ordet {ordet} forekommer {ordliste[ordet]} antall
ganger")
```

Klasser og objekter

Klassen Punkt

```
import math

class Punkt:
    # Konstruktør
```

```
    def __init__(self, start_x=0, start_y=0):
        self.__x_koordinat = start_x
        self.y_koordinat = start_y

    # Eksempel på bruk av @property og @setter for å lage en
    # egenskap med kontroll på lovlige verdier
    @property
    def x_koordinat(self):
        return self.__x_koordinat

    @x_koordinat.setter
    def x_koordinat(self, ny_verdi):
        if ny_verdi < 0:
            raise ValueError("Punktet kan ikke ha negativt x-
koordinat!")
        self.__x_koordinat = ny_verdi

    # Eksempel på en beregnet egenskap: r
    @property
    def r(self):
        return self.avstand_origo()

    @r.setter
    def r(self, ny_verdi):
        theta = self.theta
        self.x_koordinat = ny_verdi*math.cos(theta)
        self.y_koordinat = ny_verdi*math.sin(theta)

    # Eksempel på en beregnet egenskap: Theta
    @property
    def theta(self):
       return math.acos(self.x_koordinat/self.r)

    @theta.setter
    def theta(self, ny_verdi):
        r = self.r
        self.x_koordinat = r*math.cos(ny_verdi)
        self.y_koordinat = r*math.sin(ny_verdi)
```

```
    # Mutator
    def flytt(self, delta_x, delta_y):
        self.x_koordinat += delta_x
        self.y_koordinat += delta_y

    # Query
    def avstand_origo(self):
        return (self.x_koordinat**2 + self.y_koordinat**2)**0.5

    # Avstand mellom to punkter, antar at det som kommer inn er
    # et Punkt-objekt
    def avstand(self, annet_punkt):
        xdiff = self.x_koordinat - annet_punkt.x_koordinat
        ydiff = self.y_koordinat - annet_punkt.y_koordinat
        return (xdiff**2 + ydiff**2)**0.5

    # Gir en strengrepresentasjon av objektet som skal gi mening for
    # en bruker
    def __str__(self):
        return f"Punkt: ({self.x_koordinat}, {self.y_koordinat})"

    # Gir en strengrepresentasjon til mer internt bruk. Du skal være
    # i stand til å rekonstruere objektet fra denne
    def __repr__(self):
        return f"Punkt ({self.x_koordinat}, {self.y_koordinat})"


# Eksempel på en funksjon som tar inn to punkter: Regner ut
# avstanden mellom dem. Denne gjør det samme som avstand-metoden i
# klassen Punkt og viser en annen måte å formulere samme funksjon
# på.
def avstand_punkter(punkt1, punkt2):
    xdiff = punkt1.x_koordinat - punkt2.x_koordinat
    ydiff = punkt1.y_koordinat - punkt2.y_koordinat
    return (xdiff**2 + ydiff**2)**0.5


# Eksempel på en funksjon som modifiserer punktene den får inn
```

```python
def flytt_til_midten(punkt1, punkt2):
    snitt_x = (punkt1.x_koordinat + punkt2.x_koordinat)/2
    snitt_y = (punkt1.y_koordinat + punkt2.y_koordinat)/2
    punkt1.x_koordinat = snitt_x
    punkt1.y_koordinat = snitt_y
    punkt2.x_koordinat = snitt_x
    punkt2.y_koordinat = snitt_y


if __name__ == "__main__":
    punkt1 = Punkt(3, 4)
    print(punkt1)
    print(punkt1.avstand_origo())
    punkt2 = Punkt()
    print(punkt2)
    print(punkt2.avstand_origo())
    punkt1.flytt(2, -1)
    print(punkt1)
    print(punkt1.avstand_origo())
    punkt3 = Punkt(5, 10)
    avstanden = punkt1.avstand(punkt3)
    print(f"Avstanden mellom {punkt1} og {punkt3} er {avstanden}")
```

Question 2
Attached



```python
# Forslag 1

skjermstorrelse_str = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse = float(skjermstorrelse_str)
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse/nevner
bredde = 16*faktor
hoyde = 9*faktor
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")

# Forslag 2
skjermstorrelse_str = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse = float(skjermstorrelse_str)
bredde = 16*faktor
hoyde = 9*faktor
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse/nevner
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")

# Forslag 3
skjermstorrelse_str = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse = float(skjermstorrelse_str)
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse/nevner
hoyde = 9*faktor
bredde = 16*faktor
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")

# Forslag 4
skjermstorrelse = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse_float = float(skjermstorrelse)
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse_float/nevner
hoyde = 9*faktor
bredde = 16*faktor
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
```

```
    f"{hoyde} høy og {bredde} brei.")

# Forslag 5
skjermstorrelse = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse_float = float(skjermstorrelse)
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse_float/nevner
hoyde = 9*faktor
bredde = 16*faktor

# Forslag 6
skjermstorrelse = input("Skriv inn størrelsen til skjermen: ")
skjermstorrelse_float = float(skjermstorrelse)
nevner = (16**2 + 9**2)**0.5
faktor = skjermstorrelse/nevner
hoyde = 9*faktor
bredde = 16*faktor
print(f"En standard 16:9 skjerm med diagonal {skjermstorrelse} er "+
    f"{hoyde} høy og {bredde} brei.")
```

Question 4

Attached



```python
# Del 1
liste = list()
for i in range(5):
    liste.append(2*i)
print(liste)

# Del 2
liste = list()
liste.append(1)
liste.append(1)
for i in range(2, 5):
    liste.append(liste[i-1] + liste[i-2])
print(liste)
```

Question 5
Attached



```python
# Deloppgave 1

# Alternativ 1
def les_positivt_heltall1(beskjed):
    tall = int(input(beskjed))
    while tall < 0:
        print("Du må skrive inn et ikke-negativt heltall!")
        tall = int(input(beskjed))
    return tall

# Alternativ 2
def les_positivt_heltall1(beskjed):
    tall = int(input(beskjed))
    while tall < 0:
        print("Du må skrive inn et ikke-negativt heltall!")
    tall = int(input(beskjed))
    return tall


# Alternativ 3
def les_positivt_heltall1(beskjed):
    streng = input(beskjed)
    tall = int(streng)
    while tall < 0:
        print("Du må skrive inn et ikke-negativt heltall!")
        streng = input(beskjed)
        tall = int(streng)
    return tall

# Alternativ 4
def les_positivt_heltall1(beskjed):
    fortsetter = True
    while fortsetter:
        tall = int(input(beskjed))
        if tall < 0:
            print("Du må skrive inn et ikke-negativt heltall!")
        else:
            fortsetter = False
    return tall
```

```python
# Deloppgave 2

# Alternativ 1
def les_positivt_heltall_med_eh(beskjed):
    fortsetter = True
    while fortsetter:
        try:
            tall = int(input(beskjed))
            fortsetter = False
        except ValueError:
            print("Du må skrive inn et heltall!")
            continue
        if tall < 0:
            print("Du må skrive inn et ikke-negativt heltall!")
    return tall

# Alternativ 2
def les_positivt_heltall_med_eh(beskjed):
    fortsetter = True
    while fortsetter:
        try:
            tall = int(input(beskjed))
        except ValueError:
            print("Du må skrive inn et heltall!")
            continue
    if tall < 0:
        print("Du må skrive inn et ikke-negativt heltall!")
    else:
        fortsetter = False
    return tall

# Alternativ 3
def les_positivt_heltall_med_eh(beskjed):
    fortsetter = True
    while fortsetter:
        try:
            tall = int(input(beskjed))
```

```
        except ValueError:
            print("Du må skrive inn et heltall!")
            continue
        if tall < 0:
            print("Du må skrive inn et ikke-negativt heltall!")
            fortsetter = False
    return tall

# Alternativ 4
def les_positivt_heltall_med_eh(beskjed):
    fortsetter = True
    while fortsetter:
        try:
            tall = int(input(beskjed))
        except ValueError:
            print("Du må skrive inn et heltall!")
            continue
        if tall < 0:
            print("Du må skrive inn et ikke-negativt heltall!")
        else:
            fortsetter = False
    return tall
```

Question 6
Attached



```python
# Deloppgave 1, foreslått kode
tid_i_sekunder = 0


def tid_aa_kjore(avstand_i_meter, fart_i_kilometer_per_time):
    tid_i_sekunder = avstand_i_meter / (fart_i_kilometer_per_time*1000/3600)


avstand_i_meter = float(input("Avstand i meter: "))
fartsgrense = float(input("Fartsgrense i km/t: "))
tid_aa_kjore(avstand_i_meter, fartsgrense)
print(tid_i_sekunder)


# Deloppgave 2 , foreslått kode
def med_i_begge(liste1, liste2):
    resultat = list()
    for element in liste1:
        if element in liste2:
            resultat.append(element)
    return resultat


# Deloppgave 3 , foreslått kode
def sum_liste(liste1, liste2):
    resultat = list()
    for i in range(len(liste1)):
        resultat.append(liste1[i] + liste2[i])
    return resultat


# Deloppgave 4 , foreslått kode

# Gitt, du kan anta at denne er korrekt
class Punkt:
    def __init__(self, x_koordinat, y_koordinat):
        self.x_koordinat = x_koordinat
        self.y_koordinat = y_koordinat
```

```python
# Gitt, du kan anta at denne er korrekt
class Linje:
    def __init__(self, startpunkt, sluttpunkt):
        self.punktliste = list()
        self.punktliste.append(startpunkt)
        self.punktliste.append(sluttpunkt)

    def legg_til_punkt(self, punkt):
        self.punktliste.append(punkt)


# Forslag til løsning
def les_linje_fra_WKT(wkt_streng):
    if wkt_streng.find('LINESTRING') == -1:
        return None
    punktliste = list()
    punktliste_streng = wkt_streng[11:-1].split(',')
    for punkt in punktliste_streng:
        koordinater = punkt.split(' ')
        x_koordinat = float(koordinater[0])
        y_koordinat = float(koordinater[1])
        punktliste.append(Punkt(x_koordinat, y_koordinat))
    return Linje(punktliste[0], punktliste[1])
```